

시스템프로그래밍

Assignment 2-2

과제 보고서

학과: 컴퓨터정보공학부

담당교수: 최상호 교수님

수강코드: B (이론2 금12)

학번: 2024402055

성명: 박현지

1. Introduction

본 과제는 socket programming과 process 제어를 이용하여 간단한 FTP 형태의 client, server 프로그램을 구현하는 것을 목표로 한다. client는 사용자로부터 문자열을 입력받아 server로 전송하고, server는 해당 문자열을 그대로 다시 전송하는 echo 기능을 수행한다. 이를 통해 TCP 기반 socket 통신의 기본 동작 원리를 이해할 수 있다.

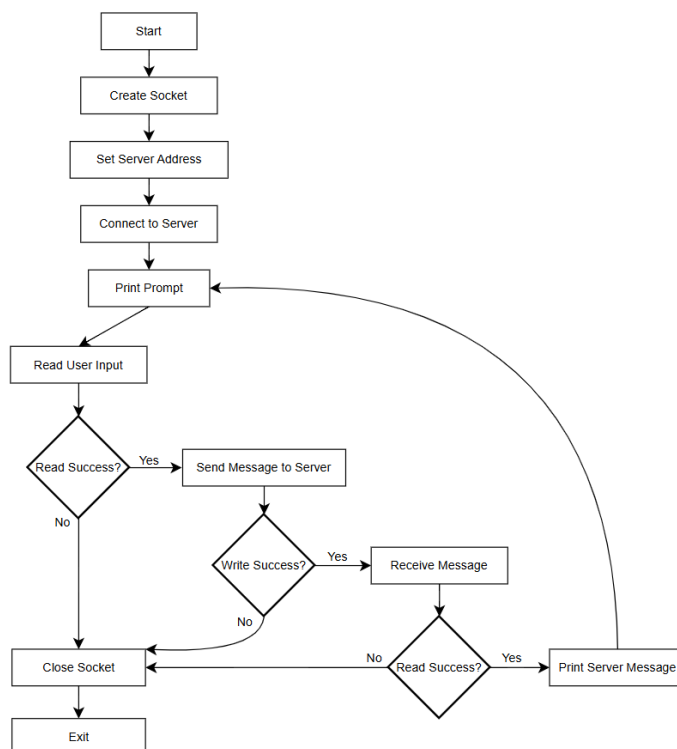
특히 server 측에서는 fork()를 이용하여 client 연결마다 새로운 child process를 생성하도록 구현하였다. parent process는 새로운 client 연결을 계속 대기하고, child process는 실제 client와 데이터를 송수신하는 구조로 동작한다. 이를 통해 multi-process 기반 서버 구조를 이해할 수 있었다.

또한 signal 처리 기능을 이용하여 SIGALRM과 SIGCHLD를 처리하였다. client가 "QUIT" 문자열을 전송하면 child process는 SIGALRM signal을 발생시켜 종료되며, parent process는 SIGCHLD signal을 처리하여 종료된 child process의 상태를 회수한다. 이를 통해 zombie process 처리 방법과 signal handler 등장 방식 또한 학습할 수 있었다.

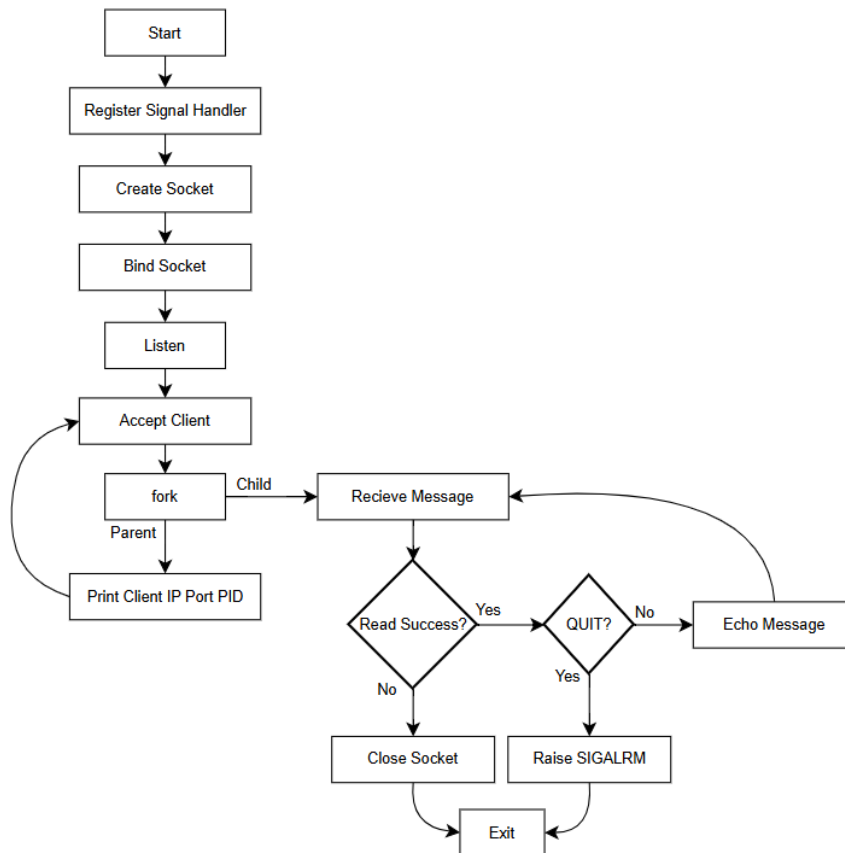
본 과제를 수행하며 socket(), bind(), listen(), accept(), fork(), signal(), read(), write()와 같은 시스템 콜의 사용법을 익힐 수 있었으며, 프로세스 생성 및 네트워크 통신 흐름에 대한 이해를 높일 수 있었다.

2. Flow Chart

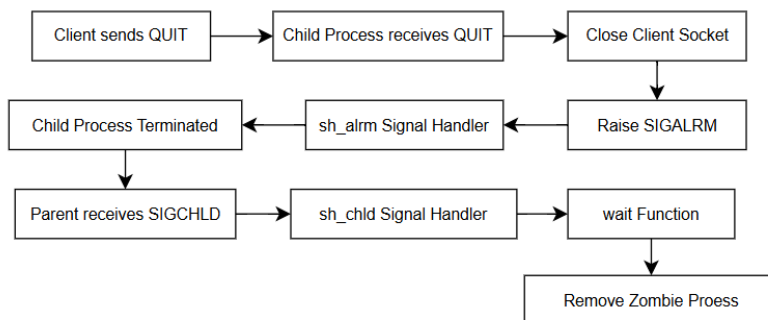
A. Client Main Flow



B. Server Main Flow



C. Signal Handling Flow



3. 결과 화면

프로그램을 컴파일한 후 다양한 옵션을 입력하여 실행 결과를 확인하였다.

A. Server 실행, Client 연결

i. Server

```

./srv 10000
kw2024402055@kw2024402055:~/26-1_System_Programming_FTP-assignment1-1/26-1_System_Programming_FTP-assignment_2-2$ ./srv 10000

```

server program을 실행한 뒤 client 연결을 대기하였다.

ii. Client

```
./cli 127.0.0.1 10000
kw2024402055@kw2024402055:~/26-1_System_Programming_FTP-assignment1-1/26-1_System_Programming/FTP-assignment_2-2$ ./srv 10000
=====Client info=====
client IP : 127.0.0.1
client port : 42526
=====
Child Process ID : 5521
kw2024402055@kw2024402055:~/26-1_System_Programming_FTP-assignment1-1/26-1_System_Programming/FTP-assignment_2-2$ ./cli 127.0.0.1 10000
>
```

client program 실행 시 server에서 client IP와 port 번호를 출력하였으며, 새로운 child process의 PID 또한 함께 출력하였다.

B. 문자열 송수신 결과

```
kw2024402055@kw2024402055:~/26-1_System_Programming_FTP-assignment1-1/26-1_System_Programming/FTP-assignment_2-2$ ./cli 127.0.0.1 10000
> this is test
from server: this is test
> 2024402055
from server: 2024402055
>
```

client에서 문자열을 입력하면 server가 해당 문자열을 그대로 다시 전송하는 echo 기능을 확인하였다.

C. QUIT 결과

i. Client

```
> QUIT
kw2024402055@kw2024402055:~/26-1_System_Programming_FTP-assignment1-1/26-1_System_Programming/FTP-assignment_2-2$
```

Client에서 "QUIT" 문자열 입력 시 child process가 SIGALRM signal에 의해 종료되며, parent process에서 SIGCHLD signal을 처리하는 것을 확인하였다.

ii. Server

```
Child Process(PID : 5521) will be terminated.
Status of child process was changed.
```

이를 통해 signal handler와 zombie process 제거 과정이 정상적으로 수행됨을 확인하였다.

D. 다중 Client 연결 결과

```
kw2024402055@kw2024402055:~/26-1_System_Programming_FTP-assignment1-1/26-1_System_Programming/FTP-assignment_2-2$ ./srv 5000
=====Client info=====
client IP : 127.0.0.1
client port : 55406
=====
Child Process ID : 5696
=====Client info=====
client IP : 127.0.0.1
client port : 44510
=====
Child Process ID : 5705
kw2024402055@kw2024402055:~/26-1_System_Programming_FTP-assignment1-1/26-1_System_Programming/FTP-assignment_2-2$ ./cli 127.0.0.1 5000
>
kw2024402055@kw2024402055:~/26-1_System_Programming_FTP-assignment1-1/26-1_System_Programming/FTP-assignment_2-2$ ./cli 127.0.0.1 5000
>
```

여러 개의 client를 동시에 실행하였을 때 각각 새로운 child process가 생성되는 것을 확인하였다. 이를 통해 fork() 기반 multi-processes server 구조가 정상적으로 동작함을 확인하였다.

4. 고찰

이번 과제를 통해 TCP socket programming의 기본 동작 방식과 client, server 구조를 직접 구현해 볼 수 있었다. 단순히 이론으로만 학습하던 socket(), bind(), listen(), accept() 함수들을 실제 프로그램에 적용하면서 네트워크 통신 흐름을 보다 명확하게 이해할 수 있었다.

특히 server에서 fork()를 이용하여 client마다 새로운 child process를 생성하는 과정은 multi-process 기반 서버 구조를 이해하는 데 큰 도움이 되었다. parent process는 지속적으로 새로운 client 연결을 대기하고, child process는 실제 client와 통신을 수행하는 구조를 구현하면서 프로세스 분리의 필요성과 장점을 경험할 수 있었다.

또한 signal handler를 이용하여 SIGALRM과 SIGCHLD를 처리하면서 process 종료 과정과 zombie processes 처리 방식 또한 학습할 수 있었다. 단순히 process를 종료하는 것이 아니라, parent process에서 wait()를 호출하여 child process의 상태를 회수해야 한다는 점을 이해할 수 있었다.

문자열 송수신 기능을 구현하면서 read()와 write()를 이용한 입출력 처리 방식 또한 익힐 수 있었다. 특히 socket 통신에서는 입력과 출력이 항상 정상적으로 수행된다는 보장이 없기 때문에, read, write 실패 여부를 반드시 확인해야 한다는 점을 배울 수 있었다.

이번 과제를 통해 시스템 프로그래밍에서 중요한 socket 통신, process 생성, signal 처리, 입출력 제어에 대한 전반적인 이해를 얻을 수 있었다. 다만 현재 구현은 단순 echo 기능만 수행하므로, 추후 실제 FTP 명령 처리나 thread 기반 서버 구조로 확장해 볼 필요가 있다고 느꼈다.

5. Reference

System Programming 강의자료 FTP 2-2